

Resolución de Sistemas de Ecuaciones No Lineales

mediante el Método de Newton-Raphson

Análisis Numérico para Ingeniería — CE-1111

Grupo 02

12 de marzo de 2026

1. Descripción del problema

Se resuelven tres sistemas de ecuaciones no lineales de la forma $F(\mathbf{x}) = \mathbf{0}$ mediante el método de Newton-Raphson para sistemas, usando tolerancia $\text{tol} = 10^{-5}$ y un máximo de 1000 iteraciones.

2. Estrategia de implementación

El método de Newton-Raphson para sistemas resuelve iterativamente

$$J(\mathbf{x}^{(k)}) \mathbf{y}^{(k)} = F(\mathbf{x}^{(k)}), \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{y}^{(k)}, \quad (1)$$

donde J es la matriz Jacobiana de F . La implementación sigue los pasos descritos a continuación.

1. **Declaración simbólica.** Las variables del sistema se declaran como símbolos usando `sym()`.
2. **Cálculo del Jacobiano.** La matriz Jacobiana $J_{ij} = \partial f_i / \partial x_j$ se construye aplicando `diff(expr, x_sym{j})` sobre cada función del sistema, obteniéndose una matriz simbólica $n \times n$.
3. **Evaluación numérica en cada iteración.** En cada paso k :
 - Se evalúa el vector $F(\mathbf{x}^{(k)})$ sustituyendo los valores actuales con `subs()`.
 - Se registra el error $\|F(\mathbf{x}^{(k)})\|_2$.
 - Si el error es menor que `tol`, se detiene.
 - Se evalúa $J(\mathbf{x}^{(k)})$ numéricamente.

- Se resuelve el sistema lineal $J\mathbf{y} = F$ con `mldivide` (operador `\`), *sin calcular la inversa de J* .
 - Se actualiza $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{y}$.
4. **Registro del error.** El vector `er_k` acumula $\|F(\mathbf{x}^{(k)})\|_2$ en cada iteración para graficar la convergencia.

3. Paquetes y bibliotecas utilizados

- `pkg load symbolic`: carga el paquete de cálculo simbólico de GNU Octave (basado en SymPy).
- `sym()`: convierte cadenas de texto a expresiones simbólicas.
- `diff(expr, var)`: diferenciación simbólica para construir el Jacobiano.
- `subs(expr, var, val)`: sustitución numérica de variables simbólicas en cada iteración.
- `double()`: convierte el resultado simbólico a valor numérico de punto flotante.
- `mldivide (\)`: resuelve el sistema lineal $J\mathbf{y} = F$ de forma eficiente sin invertir la matriz.
- `norm(v, 2)`: norma euclidiana del vector residual.
- `loglog, xlabel, ylabel, title, grid`: generación de la gráfica de convergencia en escala log-log.
- `warning('off', 'all')`: suprime advertencias del paquete simbólico durante la ejecución.

4. Valores iniciales y resultados

Primer sistema

$$\begin{cases} x_1^2 - 2x_1 - x_2 = -0,5 \\ x_1^2 + 4x_2^2 = 4 \end{cases} \quad \mathbf{x}^{(0)} = (3, 2)^T$$

Segundo sistema

$$\begin{cases} \sin(x_1) + x_2 \cos(x_1) = 0 \\ x_1 - x_2 = 0 \end{cases} \quad \mathbf{x}^{(0)} = (1, 2, -1, 5)^T$$

Tercer sistema

$$\begin{cases} x_2x_3 + x_4(x_2 + x_3) = 0 \\ x_1x_3 + x_4(x_1 + x_3) = 0 \\ x_1x_2 + x_4(x_1 + x_2) = 0 \\ x_1x_2 + x_1x_3 + x_2x_3 = 1 \end{cases} \quad \mathbf{x}^{(0)} = (-1, -1, -1, -1)^T$$

La tabla 1 presenta los resultados obtenidos para los tres sistemas.

Cuadro 1: Resultados del método de Newton-Raphson para los tres sistemas.

Sistema	Solución aproximada	Error final $\ F(\mathbf{x}^{(k)})\ _2$	Iteraciones
Sistema 1	$x_1 \approx 1,900677, x_2 \approx 0,311219$	$2,09 \times 10^{-6}$	6
Sistema 2	$x_1 \approx 0, x_2 \approx 0$	$2,51 \times 10^{-8}$	5
Sistema 3	$x_1 \approx -0,577350, x_2 \approx -0,577350$ $x_3 \approx -0,577350, x_4 \approx 0,288675$	$5,53 \times 10^{-7}$	5

5. Gráficas de convergencia

Las figuras 1–3 muestran la evolución del error $\|F(\mathbf{x}^{(k)})\|_2$ en función del número de iteración, en escala log-log, para cada sistema.

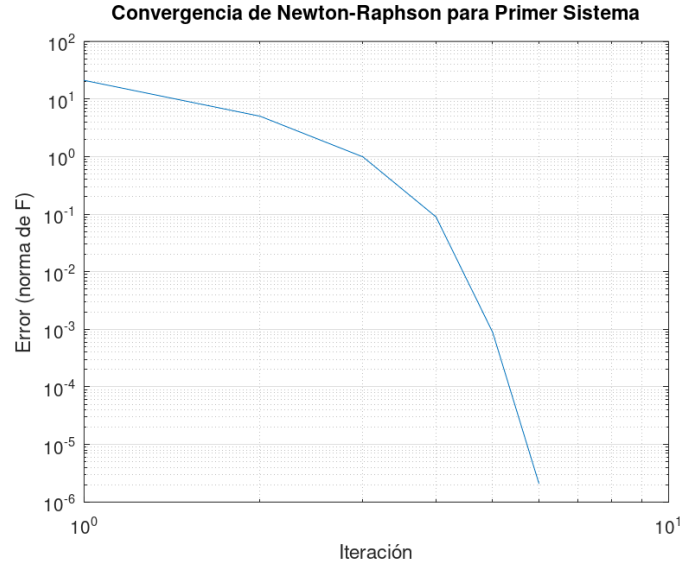


Figura 1: Convergencia de Newton-Raphson — Sistema 1.

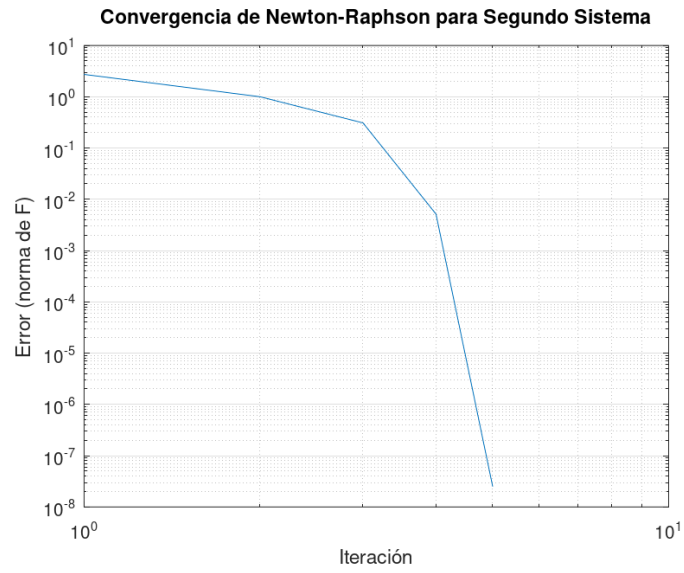


Figura 2: Convergencia de Newton-Raphson — Sistema 2.

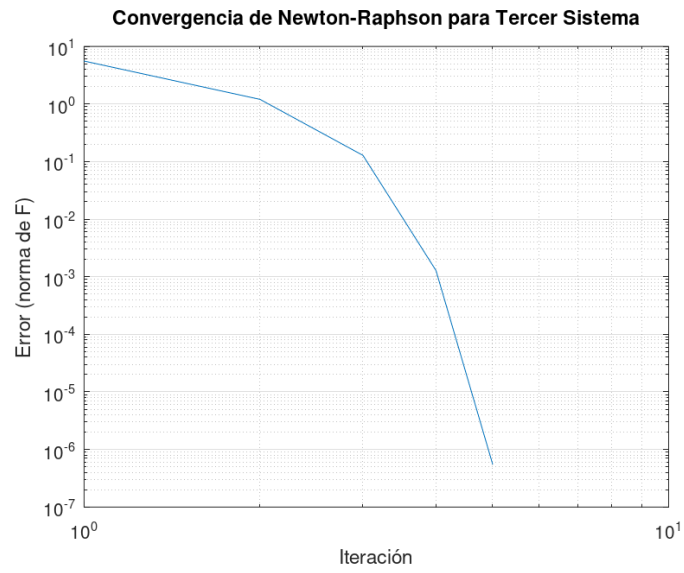


Figura 3: Convergencia de Newton-Raphson — Sistema 3.